Documento técnico

🎮 Aprendo Fácil Jugando 🎮

Guía del código: qué hay, dónde está y cómo funciona

Luis Alberto López Hernández · Carné 5990-18-1186 · Desarrollo Web 2026 · UMG

1. Resumen y mapa del proyecto

Aprendo Fácil Jugando es una aplicación web con tres capas separadas: un frontend en React, una API REST en Express (Node.js) y una base de datos MySQL. Este documento sirve de guía para quien revise el código: indica en qué archivo está cada cosa y para qué sirve.

```
ProyectoFinal/
├── README.md            Instrucciones de instalación y ejecución
├── database.sql         Script que crea la base, las tablas y las 9 actividades
├── .gitignore           Evita subir node_modules, .env y dist
├── docs/                Informe y documento técnico
├── backend/
│   ├── server.js        Punto de entrada: CORS, JSON, rutas y errores
│   ├── config/db.js     Pool de conexiones a MySQL (con SSL opcional)
│   ├── controllers/     estudiantes.js · actividades.js · progreso.js
│   ├── routes/index.js  Direcciones de la API
│   └── .env.example     Plantilla de variables de entorno
└── frontend/
    ├── index.html       Página base
    ├── .env.example     Plantilla (VITE_API_URL)
    └── src/
        ├── main.jsx · App.jsx · App.css · api.js
        ├── components/  Inicio · Registro · Menu · Juegos · Memoria · Tablero
        ├── data/preguntas.js  Generador de preguntas (aleatorización)
        └── img/         Imágenes del menú y del fondo
```

Dónde está cada cosa

Qué buscas	Dónde está	Detalle
Arranque del servidor	backend/server.js	Activa CORS, express.json(), rutas bajo /api y manejador de errores
Conexión a la base de datos	backend/config/db.js	mysql2/promise con pool de 10 conexiones; datos desde variables de entorno
Direcciones de la API	backend/routes/index.js	5 rutas: 2 de estudiantes, 1 de actividades, 2 de progreso
Registro y validación del niño	backend/controllers/estudiantes.js	Valida nombre y edad (3 a 12) antes del INSERT
Catálogo de actividades	backend/controllers/actividades.js	SELECT con filtro opcional ?nivel=
Guardar y consultar partidas	backend/controllers/progreso.js	guardar (INSERT con validación) y listar (JOIN, filtro y orden)
Estructura de la base de datos	database.sql	Tablas, llaves foráneas y datos de las 9 actividades
Pantalla que se muestra	frontend/src/App.jsx	Tres estados (rol, estudiante, actividad) deciden el componente
Llamadas al backend	frontend/src/api.js	Todas las funciones fetch; usa VITE_API_URL
Inicio: estudiante o docente	components/Inicio.jsx	Dos botones
Registro del niño	components/Registro.jsx	Formulario con validación y mensajes
Menú con imágenes	components/Menu.jsx	Pide actividades a la API; objeto IMAGENES asocia cada una con su imagen
Juegos de preguntas	components/Juegos.jsx	Muestra la pregunta, evalúa, suma puntos y guarda la partida
Juego de memoria	components/Memoria.jsx	Baraja 12 cartas, compara parejas y guarda el resultado
Tablero del docente	components/Tablero.jsx	Selector de estudiante, tarjetas de resumen y tabla
Preguntas y aleatoriedad	frontend/src/data/preguntas.js	Funciones barajar, azar y generarPreguntas
Estilos	frontend/src/App.css	Colores, fuente Fredoka, tarjetas, botones, tablero y fondo
Imágenes	frontend/src/img/	Se importan en Menu.jsx; el fondo se usa desde App.css
Variables de entorno	backend/.env y frontend/.env	No se suben a Git; se entregan como .env.example

2. Plataformas utilizadas

Pieza	Plataforma	Datos relevantes
Código fuente	GitHub (repositorio privado)	Cuenta LALopez84, repositorio aprendo-facil-jugando, rama main. Render publica desde aquí.
Base de datos en la nube	Aiven for MySQL, plan gratuito	Servicio aprendo-facil-db, MySQL 8.4, DigitalOcean (Nueva York), 1 GB, conexión SSL. El plan gratuito puede apagarse por inactividad.
Base de datos local	MySQL 8.0.46 y MySQL Workbench	localhost, puerto 3306, base aprendo_facil.
Backend en la nube	Render Web Service (gratis)	Región Virginia, Node 24, comando node server.js. Se duerme tras unos 15 minutos sin uso.
Frontend en la nube	Render Static Site (gratis)	Compilado con Vite (npm run build) y servido desde la carpeta dist.
Desarrollo	VS Code, Node.js 24, Vite, Chrome DevTools	Puertos locales: 5173 (frontend) y 10001 (backend).

Por qué dos proveedores: Render gratuito ofrece servicios web, sitios estáticos y bases PostgreSQL, pero no MySQL. Como el enunciado exige MySQL, la base de datos está en Aiven y el resto en Render. Esto además refuerza la separación del RNF-05.

3. Flujo de datos

Acción	Recorrido
Registrar un niño	Registro.jsx valida → api.js crearEstudiante → POST /api/estudiantes → estudiantes.crear valida → INSERT en MySQL → responde con el id → App.jsx guarda al estudiante en su estado.
Mostrar el menú	Menu.jsx → api.js getActividades(nivel) → GET /api/actividades?nivel=... → SELECT → JSON → un botón por actividad.
Jugar y guardar	Juegos.jsx genera preguntas con preguntas.js → el niño responde (verde/rojo) → al terminar, useEffect llama guardarProgreso → POST /api/progreso → progreso.guardar valida → INSERT.
Ver el reporte	Tablero.jsx → getProgreso(filtro) → GET /api/progreso?estudiante_id=... → SELECT con JOIN → tabla y tarjetas.

4. Base de datos

Está en database.sql. Se ejecuta una vez en MySQL (local con Workbench, o en Aiven con una conexión SSL). Las tablas son estudiantes, actividades y progreso. La tabla progreso tiene dos llaves foráneas, por lo que MySQL rechaza resultados de estudiantes o actividades inexistentes (RNF-04).

```
CREATE TABLE IF NOT EXISTS progreso (
  id INT AUTO_INCREMENT PRIMARY KEY,
  estudiante_id INT NOT NULL,
  actividad_id INT NOT NULL,
  puntaje INT NOT NULL,
  aciertos INT NOT NULL,
  intentos INT NOT NULL,
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (estudiante_id) REFERENCES estudiantes(id),
  FOREIGN KEY (actividad_id) REFERENCES actividades(id)
);
```

Dónde vive cada copia: en mi equipo, MySQL local (localhost:3306, base aprendo_facil); en producción, el servicio de Aiven. El backend decide a cuál conectarse solo por las variables de entorno DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME y DB_SSL; el código es el mismo en los dos casos.

5. Backend: el código más importante

5.1 server.js

Crea la aplicación, activa CORS solo para el origen indicado en `FRONTEND_URL`, habilita la lectura de JSON, monta las rutas en `/api` y define un manejador de errores que responde con un mensaje genérico.

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');

const app = express();
app.use(cors({ origin: process.env.FRONTEND_URL }));
app.use(express.json());
app.get('/', (req, res) => {
  res.json({ nombre: 'API Aprendo Fácil Jugando', estado: 'en línea',
actividades: '/api/actividades' });
});
app.use('/api', require('./routes'));
```

```
app.use((err, req, res, next) => {
    console.error(err);
    res.status(500).json({ error: 'Error del servidor' });
});

app.listen(process.env.PORT, () =>
    console.log(`API lista en http://localhost:${process.env.PORT}`)
);
```

5.2 config/db.js

Usa un pool de conexiones: en lugar de abrir una conexión por petición, reutiliza hasta 10. La opción ssl se activa con DB_SSL=true, necesario para Aiven.

```
module.exports = mysql.createPool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  ssl: process.env.DB_SSL === 'true' ? { rejectUnauthorized: false } : undefined,
  waitForConnections: true,
  connectionLimit: 10,
});
```

5.3 routes/index.js

```
router.post('/estudiantes', estudiantes.crear);
router.get('/estudiantes', estudiantes.listar);
router.get('/actividades', actividades.listar);
router.post('/progreso', progreso.guardar);
router.get('/progreso', progreso.listar);
```

5.4 controllers

Registro de estudiantes: valida y usa consulta parametrizada (los ? evitan la inyección SQL).

```
exports.crear = async (req, res) => {
  const { nombre, edad } = req.body;
  const edadNum = Number(edad);
  if (!nombre || !nombre.trim() || !Number.isInteger(edadNum) || edadNum < 3 ||
edadNum > 12) {
```

```
    return res.status(400).json({ error: 'Escribe un nombre y una edad entre 3 y
12 años' });
  }
  const [r] = await db.query(
    'INSERT INTO estudiantes (nombre, edad) VALUES (?, ?)', [nombre.trim(),
edadNum]
  );
  res.status(201).json({ id: r.insertId, nombre: nombre.trim(), edad: edadNum });
};
```

Guardar una partida, con validación de que todos los datos sean enteros no negativos:

```
exports.guardar = async (req, res) => {
  const { estudiante_id, actividad_id, puntaje, aciertos, intentos } = req.body;
  const valores = [estudiante_id, actividad_id, puntaje, aciertos, intentos];
  if (valores.some((n) => n === undefined || n === null ||
!Number.isInteger(Number(n)) || Number(n) < 0)) {
    return res.status(400).json({ error: 'Datos de progreso inválidos' });
  }
  await db.query(
    'INSERT INTO progreso (estudiante_id, actividad_id, puntaje, aciertos,
intentos) VALUES (?, ?, ?, ?, ?)',
    valores
  );
  res.status(201).json({ mensaje: 'Progreso guardado' });
};
```

Consulta del tablero: une las tres tablas para mostrar nombres en lugar de números, y agrega el filtro y el orden:

```
let sql = `SELECT p.id, e.nombre AS estudiante, a.nombre AS actividad,
           p.puntaje, p.aciertos, p.intentos, p.fecha
           FROM progreso p
           JOIN estudiantes e ON e.id = p.estudiante_id
           JOIN actividades a ON a.id = p.actividad_id`;
if (estudiante_id) {
  sql += ' WHERE p.estudiante_id = ?';
  params.push(estudiante_id);
}
sql += ' ORDER BY p.fecha DESC';
```

6. Frontend: el código más importante

6.1 App.jsx: qué pantalla se muestra

No usé librerías de rutas. Tres estados (rol, estudiante y actividad) determinan el componente que se dibuja:

```
let pantalla;
if (rol === 'docente') {
  pantalla = <Tablero onSalir={cerrarSesion} />;
} else if (rol !== 'estudiante') {
  pantalla = <Inicio onEstudiante={() => setRol('estudiante')} onDocente={() =>
setRol('docente')} />;
} else if (!estudiante) {
  pantalla = <Registro onListo={setEstudiante} onVolver={cerrarSesion} />;
} else if (!actividad) {
  pantalla = <Menu estudiante={estudiante} onElegir={setActividad}
onSalir={cerrarSesion} />;
} else if (actividad.tema === 'memoria') {
  pantalla = <Memoria key={actividad.id} estudiante={estudiante}
actividad={actividad} onSalir={volver} />;
} else {
  pantalla = <Juego key={actividad.id} estudiante={estudiante}
actividad={actividad} onSalir={volver} />;
}
```

6.2 api.js: la comunicación con el backend

Concentra todas las llamadas. La dirección base viene de la variable VITE_API_URL, de modo que el mismo código funciona en local y en la nube.

```
let pantalla;
if (rol === 'docente') {
  pantalla = <Tablero onSalir={cerrarSesion} />;
} else if (rol !== 'estudiante') {
  pantalla = <Inicio onEstudiante={() => setRol('estudiante')} onDocente={() =>
setRol('docente')} />;
} else if (!estudiante) {
  pantalla = <Registro onListo={setEstudiante} onVolver={cerrarSesion} />;
} else if (!actividad) {
  pantalla = <Menu estudiante={estudiante} onElegir={setActividad}
onSalir={cerrarSesion} />;
} else if (actividad.tema === 'memoria') {

const API = import.meta.env.VITE_API_URL;
```

```
export const crearEstudiante = (datos) => post('/estudiantes', datos);
export const getEstudiantes = () => pedir('/estudiantes');
export const getActividades = (nivel) => pedir(`/actividades?nivel=${nivel}`);
export const guardarProgreso = (datos) => post('/progreso', datos);
export const getProgreso = (estudianteId = '') =>
  pedir(`/progreso${estudianteId ? `?estudiante_id=${estudianteId}` : ''}`);
```

6.3 Juegos.jsx: evaluar, puntuar y guardar

El puntaje se calcula en vivo (RF-04). Al responder, la opción cambia de color (RF-03) y tras 1.3 segundos pasa a la siguiente pregunta. Al terminar, useEffect envía el resultado a la API (RF-05).

```
const puntaje = aciertos * 10;

const responder = (op) => {
  if (elegida) return;
  setElegida(op);
  if (op === p.correcta) setAciertos((a) => a + 1);
  setTimeout(() => { setElegida(null); setI((n) => n + 1); }, 1300);
};

useEffect(() => {
  if (fin) {
    guardarProgreso({
      estudiante_id: estudiante.id, actividad_id: actividad.id,
      puntaje, aciertos, intentos: preguntas.length,
    }).then(() => setEstado('ok')).catch(() => setEstado('error'));
  }
}, [fin]);
```

7. Aleatorización: cómo evito que siempre salga lo mismo

Toda la lógica está en frontend/src/data/preguntas.js (y en Memoria.jsx para las cartas). Se apoya en dos funciones básicas: barajar y azar.

```
const barajar = (a) => [...a].sort(() => Math.random() - 0.5);
const azar = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min;

const mezclarOpciones = (correcta, candidatos) => {
  const malas = [...new Set(candidatos)].filter((c) => c !== correcta);
  return barajar([correcta, ...barajar(malas).slice(0, 3)]);
};
```

Qué se aleatoriza	Dónde	Cómo
Orden de las preguntas espaciales	generarPreguntas → barajar(BANCO[nombre])	Cada partida baraja las 5 preguntas de la actividad.
Figuras geométricas	preguntasFiguras()	Baraja las 4 figuras repetidas dos veces (8 preguntas) y baraja también las 4 opciones.
Tablas de multiplicar	tablas()	Dos números al azar entre 2 y 10 en cada pregunta (8 preguntas).
Suma de quebrados	quebrados()	Denominadores al azar (2 a 6, y el segundo múltiplo del primero); solo acepta resultados irreducibles y menores que 1.
Multiplicación con decimales	decimales()	Cuatro preguntas decimal × entero y cuatro decimal × decimal, con números al azar.
Posición de la respuesta correcta	mezclarOpciones()	Toma 3 respuestas incorrectas al azar y baraja las 4 opciones, así la correcta cambia de lugar.
Cartas de memoria	Memoria.jsx → nuevoMazo()	Baraja las 12 cartas (6 parejas) al iniciar y al reiniciar.
Volver a jugar	Juegos.jsx → repetir()	Vuelve a llamar generarPreguntas, generando un juego nuevo.

Preguntas de matemáticas (se inventan en cada partida):

```
const tablas = () =>
  Array.from({ length: 8 }, () => {
    const a = azar(2, 10);
    const b = azar(2, 10);
    const r = a * b;
    return {
      texto: `¿Cuál es el resultado de ${a} * ${b}?`,
```

```
      opciones: numOpciones(r, [r + a, r - a, r + b, r - b, r + 1, r - 1, r +
10]),
      correcta: fmt(r),
    };
  });
```

Figuras y selección según la actividad:

```
const preguntasFiguras = () =>
  barajar([...FIGURAS, ...FIGURAS]).map((nombre) => ({
    texto: `Toca el ${nombre.toLowerCase()}`,
    opciones: barajar(FIGURAS),
    correcta: nombre,
  }));

export function generarPreguntas(actividad) {
  if (actividad.tema === 'figuras') return preguntasFiguras();
  if (actividad.tema === 'matematicas') return (MATE[actividad.nombre] ||
tablas)();
  return barajar(BANCO[actividad.nombre] || []);
}
```

Mazo de memoria:

```
const EMOJIS = ['🐶', '🐱', '🐸', '🦊', '🐼', '🐵'];
const nuevoMazo = () =>
  [...EMOJIS, ...EMOJIS].sort(() => Math.random() - 0.5).map((e, id) => ({ id, e
}));
```

Limitaciones que reconozco: (1) los juegos espaciales (adentro/afuera, arriba/abajo, cerca/lejos, encima/debajo) tienen un banco fijo de 5 preguntas, por lo que el contenido cambia de orden pero no de preguntas; (2) sort(() => Math.random() - 0.5) es suficiente para este uso, pero no es un mezclado perfectamente uniforme; la alternativa más rigurosa es el algoritmo Fisher-Yates.

8. Autenticación y control de acceso

El proyecto no tiene autenticación con usuario y contraseña. Fue una decisión de diseño por tres razones: el RF-01 pide solo "nombre y edad"; los usuarios son niños pequeños que no manejan contraseñas; y el objetivo era el seguimiento del aprendizaje, no la gestión de cuentas.

- **Estudiante:** al registrarse, MySQL genera un id que el frontend guarda en el estado de React mientras dura la sesión. Si el niño recarga la página, la sesión se pierde y debe registrarse otra vez.
- **Docente:** el botón "Soy docente" abre el tablero sin contraseña. Las rutas GET /api/estudiantes y GET /api/progreso son públicas.
- **Datos mínimos:** solo se guardan nombre y edad, sin apellidos, correos ni datos de contacto, para limitar el impacto de cualquier exposición.
- **Sin cookies ni sesiones en el servidor,** por lo que no aplican riesgos como CSRF.

Mejora propuesta: inicio de sesión para docentes con contraseña cifrada (bcrypt) y token (JWT), y proteger con él las rutas de consulta. Los estudiantes podrían seguir entrando solo con su nombre.

9. Seguridad y hardening

Apliqué **hardening básico**; no un hardening completo. Esta tabla resume honestamente qué hice y qué no:

Medida	Estado	Dónde	Comentario
Secretos fuera del código (variables de entorno)	**Aplicada**	.env, panel de Render	.env en .gitignore; se entrega .env.example
Repositorio privado	**Aplicada**	GitHub	El código viaja en el ZIP de entrega
CORS restringido a un origen	**Aplicada**	server.js	Variable FRONTEND_URL
Consultas SQL parametrizadas	**Aplicada**	controllers/	Evita inyección SQL
Validación de entradas (front y back)	**Aplicada**	Registro.jsx, estudiantes.js, progreso.js	Ids inexistentes los rechaza la llave foránea (error 500)
Errores genéricos al cliente	**Aplicada**	server.js	El detalle queda solo en el registro del servidor
HTTPS en la nube	**Aplicada**	Render y Aiven	Lo provee la plataforma
Integridad referencial	**Aplicada**	database.sql	Dos FOREIGN KEY
Pool de conexiones con límite	**Aplicada**	config/db.js	connectionLimit: 10
Conexión SSL a MySQL	**Parcial**	config/db.js	Cifrada, pero con rejectUnauthorized: false (no verifica el certificado)
Autenticación y autorización	**No aplicada**	-	Ver sección 8

Medida	Estado	Dónde	Comentario
Cabeceras de seguridad (Helmet)	**No aplicada**	-	Además la respuesta muestra X-Powered-By: Express
Límite de peticiones (rate limiting)	**No aplicada**	-	Pendiente
Usuario de BD con privilegios mínimos	**No aplicada**	Variables de Render	Se usa el usuario administrador de Aiven
Revisión de dependencias	**Parcial**	backend/package.json	npm install reporta 1 vulnerabilidad alta pendiente de revisar con npm audit

Mejoras recomendadas, por prioridad

- Login de docentes con bcrypt y JWT, y proteger las rutas de consulta.
- Agregar Helmet, desactivar X-Powered-By y limitar peticiones con express-rate-limit.
- Crear un usuario de MySQL con permisos solo sobre la base del proyecto y verificar el certificado de Aiven con su CA.
- Revisar las dependencias con npm audit.
- Ampliar los bancos de preguntas espaciales y usar Fisher-Yates para barajar.

10. Dónde van las imágenes y por qué

Las imágenes están en **frontend/src/img/** y se usan con import, por ejemplo import adentro from "../img/adentro-afuera.jpg" en Menu.jsx. Elegí esta ubicación por estas razones:

- **Vite las procesa.** Al compilar, copia cada imagen a dist/assets, les agrega un código al nombre y reescribe las rutas automáticamente. Por eso las rutas son relativas y funcionan en cualquier servidor o carpeta.
- **Si falta una imagen, el error aparece al compilar** (npm run build), no cuando el usuario abre la página.
- **Render publica solo la carpeta frontend** (Root Directory). Una carpeta img en la raíz del proyecto, fuera de frontend, no estaría disponible al compilar, y Vite no permite importar archivos fuera de su carpeta de trabajo.
- **Alternativa:** frontend/public/img/, que se copia tal cual y se usa con rutas como /img/archivo.jpg. Funciona, pero depende de que el sitio esté en la raíz del dominio, por eso preferí src/img.

El fondo se carga desde App.css con una ruta relativa: url("./img/fondo-pagina3.jpg").

11. Despliegue (resumen)

Servicio	Configuración
Aiven MySQL	Plan gratuito; usuario avnadmin; SSL obligatorio; el script database.sql se cargó con MySQL Workbench.
Render Web Service	Root Directory backend · Build npm install · Start node server.js · Variables: DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, DB_SSL=true, FRONTEND_URL. No se define PORT (lo asigna Render).
Render Static Site	Root Directory frontend · Build npm install && npm run build · Publish dist · Variable VITE_API_URL con la dirección del backend terminada en /api.

12. Limitaciones y trabajo futuro

- Sin login: el tablero del docente es público.
- Servicios gratuitos: el backend tarda hasta un minuto en despertar y Aiven puede apagar la base por inactividad.
- Los juegos espaciales tienen un banco de solo 5 preguntas cada uno.

13. Preguntas probables y respuestas

Pregunta	Respuesta
¿Tiene autenticación?	No. Por diseño: el RF-01 pide nombre y edad y los usuarios son niños. El docente entra sin contraseña; lo reconozco como limitación y propongo bcrypt y JWT.
¿Hiciste hardening?	Básico: variables de entorno, CORS restringido, consultas parametrizadas, validación, errores genéricos, HTTPS y SSL. Falta autenticación, Helmet, límite de peticiones y un usuario de BD con privilegios mínimos.
¿Cómo evitas la inyección SQL?	Con consultas parametrizadas (signos ?) en todos los controladores; los valores nunca se concatenan al texto SQL.
¿Qué es CORS y dónde lo configuraste?	Es la política del navegador que impide que una página hable con un servidor de otro origen si este no lo autoriza. Está en server.js con la opción origin tomada de FRONTEND_URL.
¿Cómo aleatorizas el contenido?	En data/preguntas.js: barajar para el orden, azar para los números de matemáticas y mezclarOpciones para que la respuesta correcta cambie de posición. Memoria baraja las cartas.
¿Por qué el primer acceso tarda?	Render gratuito apaga el backend tras unos 15 minutos sin uso y la primera petición lo despierta. Después responde en unos 250 ms.
¿Dónde está la base de datos?	En Aiven (MySQL 8.4, plan gratuito) para producción y en MySQL local para desarrollo; el código es el mismo y cambian las variables de entorno.
¿Qué pasa si el estudiante recarga la página?	Pierde la sesión (el id está en el estado de React) y debe registrarse de nuevo. Las partidas ya guardadas permanecen en la base.
¿Por qué Render y Aiven?	No hay hosting gratuito único con Node.js y MySQL 8. Render ofrece servicios web y sitios estáticos gratis, y Aiven ofrece MySQL gratuito.
¿Por qué las imágenes están en src/img?	Porque Vite las procesa al compilar y reescribe las rutas; y porque Render solo ve la carpeta frontend.
¿Cómo lo ejecuto en mi computadora?	Con el README: ejecutar database.sql en MySQL, npm install y .env en backend y frontend, node server.js y npm run dev.
¿Qué mejorarías?	Login de docentes, Helmet y rate limiting, usuario de BD con privilegios mínimos, más preguntas y pruebas automáticas.